

Chapter 10: Java Server Pages (JSP)/Servlet Technology

10.1. Applets, Servlets, and Java Server Pages

When we instruct our Web browser to view a page from a Web server on the Internet, our Web browser requests the page from the Web server, the Web server processes the request (which may involve reading the requested page from a file on the hard drive), and then the Web server sends the requested page to our Web browser. Our Web browser *formats*, or *renders*, the *received data to fit on our computer screen*. This interaction is a specific case of the **client/server** model. Our Web browser is the client program, our computer is the *client computer*, the remote website is the *server computer*, and the Web server software running on the remote website is the *server program*.

In the context of a Web application, the *client/server* model is important because Java code can run in two places: on the **client** or on **the server**. There are trade-offs to both approaches. *Server-based programs have easy access to information that resides on the server*, such as customer orders or inventory data. Because all of the computation is done on the server and results are transmitted to the client as HTML, a client does not need a powerful computer to run a server-based program. On the other hand, a *client-based* program may require a more powerful client computer, because all computation is performed locally. However, richer interaction is possible, because the client program has access to local resources, such as the *graphics display* (e.g., perhaps using Swing) or the operating system. Many systems today are constructed using code that runs on both the client and the server to reap the benefit of both approaches.

Web applications built with Java include **Java applets**, **Java servlets**, and **Java Server Pages (JSP)**. Java applets run on the *client computer*. **JavaScript**, which is a different language than Java despite its similar name, also runs on the client computer as part of the Web browser. **Java servlets** and **Java Server Pages** run on the *server*. *Java Server pages* which are a dynamic version of *Java servlets*. *Servlets must be compiled before they can run, just like a normal Java program*. In contrast, *JSP code is embedded with the corresponding HTML and is compiled “on the fly” into a servlet when the page is requested*. This flexibility can make it easier to develop Web applications using JSP than with Java servlets. The following fig1, fig2 and fig2 shows that the running a Java applet, Java Servlet and JSP program.

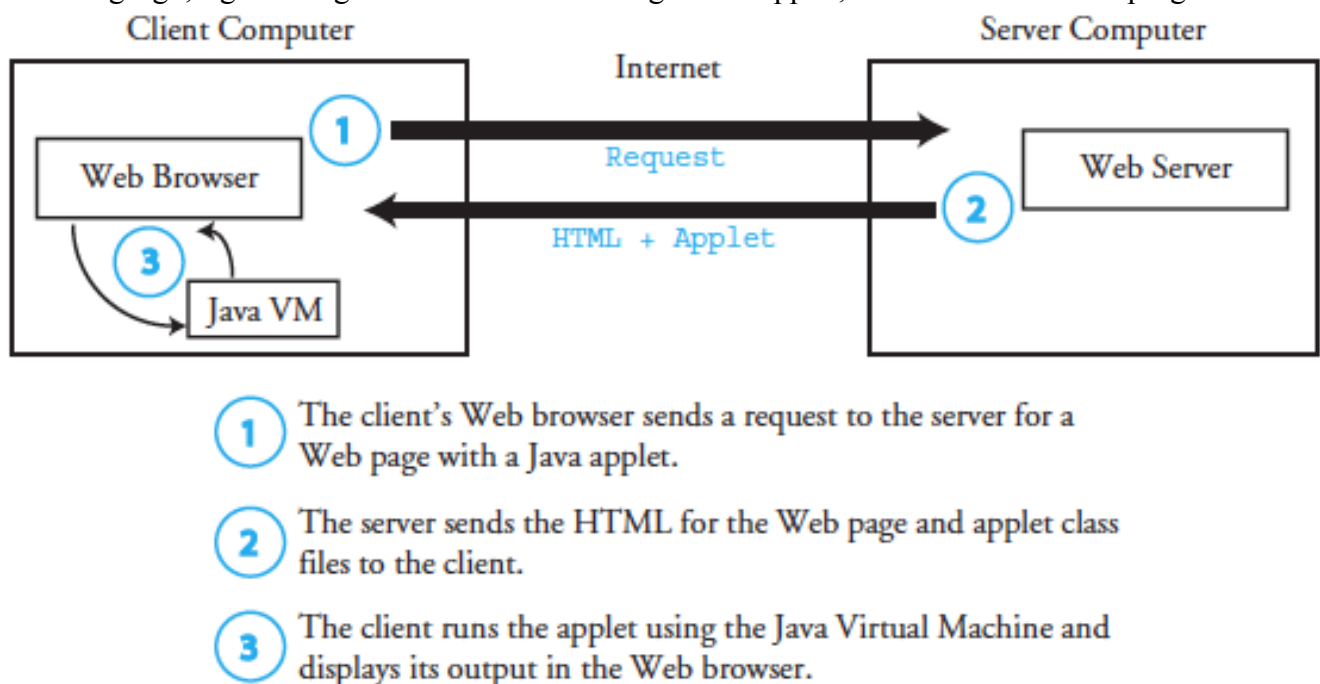


Fig1: Running a Java Applet

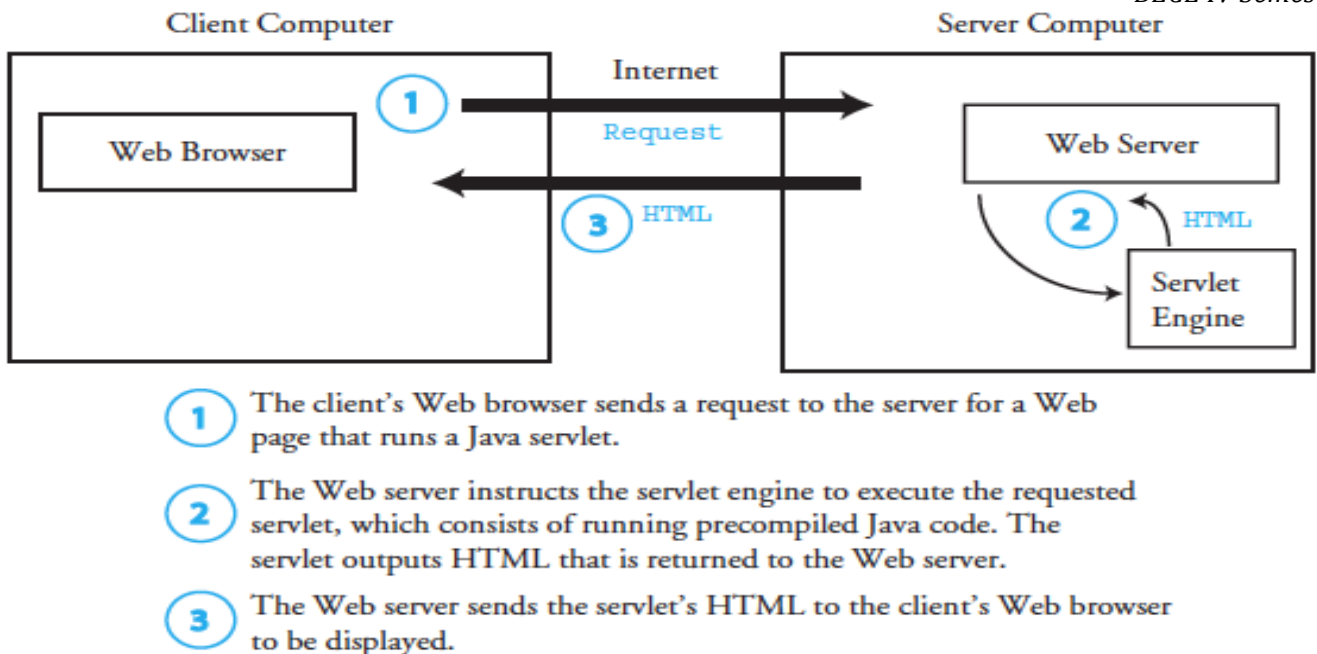


Fig2: Running a Java Servlet

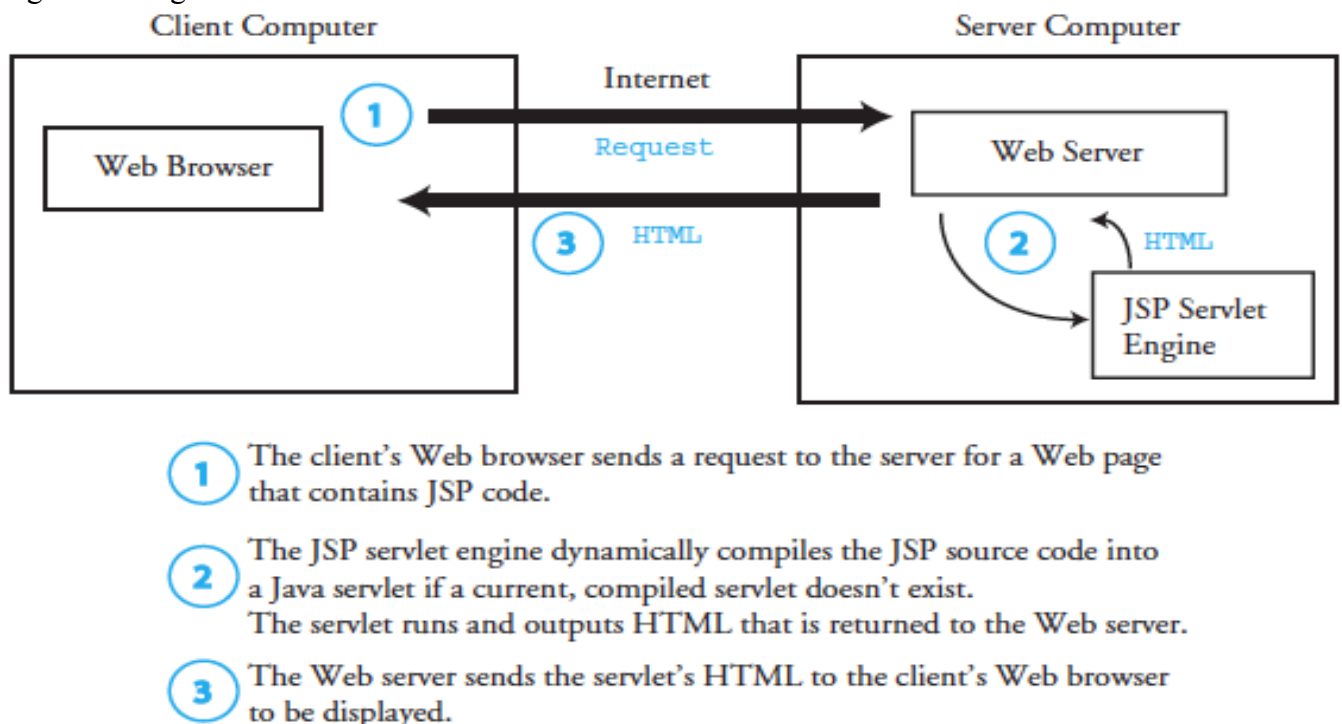


Fig3: Running a Java Server Page (JSP) Program

10.2. Servlet

Servlets offer several advantages in comparison with traditional system.

1. **First**, *performance is significantly better*. Servlets execute within the *address space* of a web server. It is *not necessary* to create a separate process to handle each client request.
2. **Second**, *servlets are platform-independent because they are written in Java*.
3. **Third**, *the Java security manager on the server enforces a set of restrictions to protect the resources on a server machine*.
4. **Finally**, *the full functionality of the Java class libraries is available to a servlet*. It can communicate with *applets, databases, or other software* via the sockets and RMI mechanisms.

10.3. The Life Cycle of a Servlet

Three methods are central to the life cycle of a servlet. These are *init()*, *service()*, and *destroy()*. They are implemented by every *servlet* and are invoked at specific times by the server. Let us consider a typical user scenario to understand when these methods are called.

1. **First**, assume that a user enters a Uniform Resource Locator (URL) to a web browser. The browser then generates an HTTP request for this URL. This request is then sent to the appropriate server.
2. **Second**, the web server receives this HTTP request. The server maps this request to a particular *servlet*. The servlet is dynamically retrieved and loaded into the address space of the server.
3. **Third**, the server invokes the *init()* method of the servlet. This method is invoked only when the servlet is first loaded into memory. It is possible to pass initialization parameters to the servlet so it may configure itself.
4. **Fourth**, the server invokes the *service()* method of the servlet. This method is called *to process the HTTP request*. It may also formulate an HTTP response for the client. The servlet remains in the *server's address space* and is available to process any other HTTP requests received from clients. The *service()* method is called for each HTTP request.
5. **Finally**, the server may decide to *unload* the servlet from its memory. The algorithms by which this determination is made are specific to each server. The server calls the *destroy()* method to relinquish any resources such as file handles that are allocated for the servlet. Important data may be saved to a persistent store. The memory allocated for the servlet and its objects can then be garbage collected.

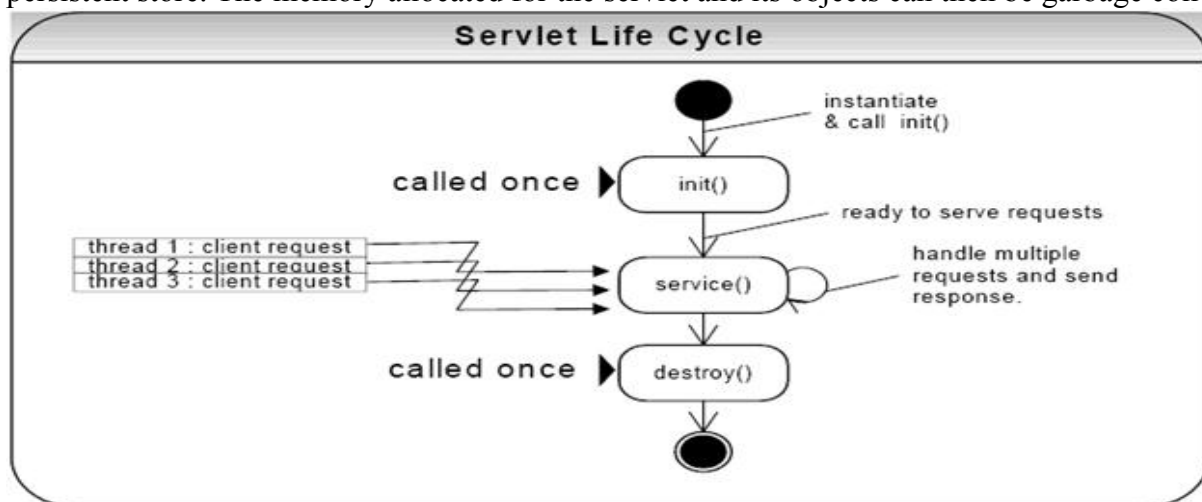


Fig4: Servlet Life cycle

10.4. A Simple Servlet

To become familiar with the key servlet concepts, we will begin by building and testing a simple servlet. The basic steps are the following:

1. Create and compile the servlet source code. Then, copy the servlet's class file to the proper directory, and add the servlet's name and mappings to the proper *web.xml* file.
2. Start Tomcat.
3. Start a web browser and request the servlet.

Let us examine each of these steps in detail.

Create and Compile the Servlet Source Code

To begin, create a file named *HelloServlet.java* that contains the following program:

```
import java.io.*;
import javax.servlet.*;
public class HelloServlet extends GenericServlet {
    public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException {
```

```
response.setContentType("text/html");
PrintWriter pw = response.getWriter();
pw.println("<B>Hello!");
pw.close();
}
```

Let's look closely at this program. First, note that it imports the **javax.servlet** package. This package contains the classes and interfaces required to build servlets. Next, the program defines **HelloServlet** as a subclass of **GenericServlet**. The **GenericServlet** class provides functionality that simplifies the *creation of a servlet*. For example, it provides versions of **init()** and **destroy()**, which may be used as is. We need supply only the **service()** method. Inside **HelloServlet**, the **service()** method (which is inherited from **GenericServlet**) is overridden. This method handles requests from a client. Notice that the first argument is a **ServletRequest** object. This enables the servlet to read data that is provided via the client request. The second argument is a **ServletResponse** object. This enables the servlet to formulate a response for the client.

The call to **setContentType()** establishes the MIME type of the HTTP response. In this program, the MIME type is **text/html**. This indicates that the browser should interpret the content as HTML source code. Next, the **getWriter()** method obtains a **PrintWriter**. Anything written to this stream is sent to the client as part of the HTTP response. Then **println()** is used to write some simple HTML source code as the HTTP response.

Compile this source code and place the **HelloServlet.class** file in the proper Tomcat directory. Also, add **HelloServlet** to the **web.xml** file.

Start Tomcat

Tomcat must be running before you try to execute a servlet.

Start a Web Browser and Request the Servlet

Start a web browser and enter the URL shown here:

<http://localhost:8080/servlets-examples/servlet/HelloServlet>

Alternatively, we may enter the URL shown here:

<http://127.0.0.1:8080/servlets-examples/servlet/HelloServlet>

This can be done because 127.0.0.1 is defined as the IP address of the local machine. We will observe the output of the servlet in the browser display area. It will contain the string **Hello!** in bold type.

10.5. The Servlet API

Two packages contain the classes and interfaces that are required to build servlets. These are **javax.servlet** and **javax.servlet.http**. They constitute the Servlet API.

10.5.1. The javax.servlet Package

The **javax.servlet** package contains a number of interfaces and classes that establish the framework in which servlets operate. The following table summarizes the core interfaces that are provided in this package. The most significant of these is **Servlet**. All servlets must implement this interface or extend a class that implements the interface. The **ServletRequest** and **ServletResponse** interfaces are also very important.

Interface	Description
Servlet	Declares life cycle methods for a servlet.
ServletConfig	Allows servlets to get initialization parameters.
ServletContext	Enables servlets to log events and access information about their environment.
ServletRequest	Used to read data from a client request.
ServletResponse	Used to write data to a client response.

The following table summarizes the core classes that are provided in the **javax.servlet** package:

Class	Description
GenericServlet	Implements the Servlet and ServletConfig interfaces.
ServletInputStream	Provides an input stream for reading requests from a client.
ServletOutputStream	Provides an output stream for writing responses to a client.
ServletException	Indicates a servlet error occurred.
UnavailableException	Indicates a servlet is unavailable.

10.5.1.1. The Servlet Interface

All servlets must implement the *Servlet interface*. It declares the **init()**, **service()**, and **destroy()** methods that are called by the server during the *life cycle of a servlet*. A method is also provided that allows a servlet to obtain any initialization parameters. The **getServletConfig()** method is called by the servlet to obtain initialization parameters. A servlet developer overrides the **getServletInfo()** method to provide a string with useful information (for example, author, version, date, copyright). The server invokes this method also.

10.5.1.2. The ServletRequest Interface

The *ServletRequest* interface enables a servlet to obtain information about a client request. Several of its methods are summarized in following table.

Method	Description
Object getAttribute(String attr)	Returns the value of the attribute named <i>attr</i> .
String getCharacterEncoding()	Returns the character encoding of the request.
int getContentLength()	Returns the size of the request. The value -1 is returned if the size is unavailable.
String getContentType()	Returns the type of the request. A null value is returned if the type cannot be determined.
ServletInputStream getInputStream() throws IOException	Returns a ServletInputStream that can be used to read binary data from the request. An IllegalStateException is thrown if getReader() has already been invoked for this request.
String getParameter(String pname)	Returns the value of the parameter named <i>pname</i> .
Enumeration getParameterNames()	Returns an enumeration of the parameter names for this request.
String[] getParameterValues(String name)	Returns an array containing values associated with the parameter specified by <i>name</i> .
String getProtocol()	Returns a description of the protocol.
BufferedReader getReader() throws IOException	Returns a buffered reader that can be used to read text from the request. An IllegalStateException is thrown if getInputStream() has already been invoked for this request.
String getRemoteAddr()	Returns the string equivalent of the client IP address.
String getRemoteHost()	Returns the string equivalent of the client host name.
String getScheme()	Returns the transmission scheme of the URL used for the request (for example, "http", "ftp").
String getServerName()	Returns the name of the server.
int getServerPort()	Returns the port number.

Table: Various methods defined by *ServletRequest*

10.5.1.3. The ServletResponse Interface

The *ServletResponse* interface enables a servlet to formulate a response for a client. Several of its

methods are summarized in following table.

Method	Description
<code>String getCharacterEncoding()</code>	Returns the character encoding for the response.
<code>ServletOutputStream getOutputStream() throws IOException</code>	Returns a ServletOutputStream that can be used to write binary data to the response. An IllegalStateException is thrown if getWriter() has already been invoked for this request.
<code>PrintWriter getWriter() throws IOException</code>	Returns a PrintWriter that can be used to write character data to the response. An IllegalStateException is thrown if getOutputStream() has already been invoked for this request.
<code>void setContentLength(int size)</code>	Sets the content length for the response to <i>size</i> .
<code>void setContentType(String type)</code>	Sets the content type for the response to <i>type</i> .

Table: Various methods defined by *ServletResponse*

10.5.2. The GenericServlet Class

The **GenericServlet** class provides implementations of the basic life cycle methods for a servlet. **GenericServlet** implements the **Servlet** and **ServletConfig** interfaces. In addition, a method to append a string to the server log file is available. The signatures of this method are shown here:

`void log(String s)`

`void log(String s, Throwable e)`

Here, *s* is the string to be appended to the log, and *e* is an exception that occurred.

10.6. Reading Servlet Parameters

The **ServletRequest** interface includes methods that allow us to read the names and values of parameters that are included in a client request. The example contains two files. A web page is defined in **PostParameters.htm**, and a servlet is defined in **PostParametersServlet.java**. The HTML source code for **PostParameters.htm** is shown in the following listing. It defines a table that contains two labels and two text fields. One of the labels is Employee and the other is Phone. There is also a submit button. Notice that the action parameter of the form tag specifies a URL. The URL identifies the servlet to process the HTTP POST request.

```
<html>
  <body>
    <center>
      <form name="Form1" method="post"
        action="http://localhost:8080/servlets-
        examples/servlet/PostParametersServlet">
        <table>
          <tr>
            <td><B>Employee</td>
            <td><input type="text" name="e" size="25" value=""></td>
          </tr>
          <tr>
            <td><B>Phone</td>
            <td><input type="text" name="p" size="25" value=""></td>
          </tr>
        </table>
        <input type="submit" value="Submit">
      </body>
</html>
```

The source code for **PostParametersServlet.java** is shown in the following listing. The **service()** method is overridden to process client requests. The **getParameterNames()** method returns an enumeration of the parameter names. These are processed in a loop. We can see that the parameter name and value are

output to the client. The parameter value is obtained via the **getParameter()** method.

```
import java.io.*;
import java.util.*;
import javax.servlet.*;
public class PostParametersServlet extends GenericServlet {
    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        // Get print writer.
        PrintWriter pw = response.getWriter();
        // Get enumeration of parameter names.
        Enumeration e = request.getParameterNames();
        // Display parameter names and values.
        while(e.hasMoreElements()) {
            String pname = (String)e.nextElement();
            pw.print(pname + " = ");
            String pvalue = request.getParameter(pname);
            pw.println(pvalue);
        }
        pw.close();
    }
}
```

Compile the servlet. Next, copy it to the appropriate directory, and update the **web.xml** file. Then, perform these steps to test this example:

1. Start Tomcat (if it is not already running).
2. Display the web page in a browser.
3. Enter an employee name and phone number in the text fields.
4. Submit the web page.

After following these steps, the browser will display a response that is dynamically generated by the servlet.

10.7. The javax.servlet.http Package

The **javax.servlet.http** package contains a number of interfaces and classes that are commonly used by servlet developers. The following table summarizes the core interfaces that are provided in this package:

Interface	Description
HttpServletRequest	Enables servlets to read data from an HTTP request.
HttpServletResponse	Enables servlets to write data to an HTTP response.
HttpSession	Allows session data to be read and written.
HttpSessionBindingListener	Notifies an object that it is bound to or unbound from a session.

The following table summarizes the core classes that are provided in this package. The most important of these is **HttpServlet**. Servlet developers typically extend this class in order to process HTTP requests.

Class	Description
Cookie	Allows state information to be stored on a client machine.
HttpServlet	Provides methods to handle HTTP requests and responses.
HttpSessionEvent	Encapsulates a session-changed event.
HttpSessionBindingEvent	Indicates when a listener is bound to or unbound from a session value, or that a session attribute changed.

10.7.1. The HttpServletRequest Interface

The **HttpServletRequest** interface enables a servlet to obtain information about a client request. Several of its methods are shown in following table.

Method	Description
String getAuthType()	Returns authentication scheme.
Cookie[] getCookies()	Returns an array of the cookies in this request.
long getDateHeader(String field)	Returns the value of the date header field named <i>field</i> .
String getHeader(String field)	Returns the value of the header field named <i>field</i> .
Enumeration getHeaderNames()	Returns an enumeration of the header names.
int getIntHeader(String field)	Returns the int equivalent of the header field named <i>field</i> .
String getMethod()	Returns the HTTP method for this request.
String getPathInfo()	Returns any path information that is located after the servlet path and before a query string of the URL.
String getPathTranslated()	Returns any path information that is located after the servlet path and before a query string of the URL after translating it to a real path.
String getQueryString()	Returns any query string in the URL.
String getRemoteUser()	Returns the name of the user who issued this request.
String getRequestedSessionId()	Returns the ID of the session.
String getRequestURI()	Returns the URL.
StringBuffer getRequestURL()	Returns the URL.
String getServletPath()	Returns that part of the URL that identifies the servlet.
HttpSession getSession()	Returns the session for this request. If a session does not exist, one is created and then returned.
HttpSession getSession(boolean new)	If <i>new</i> is true and no session exists, creates and returns a session for this request. Otherwise, returns the existing session for this request.
boolean isRequestedSessionIdFromCookie()	Returns true if a cookie contains the session ID. Otherwise, returns false .
boolean isRequestedSessionIdFromURL()	Returns true if the URL contains the session ID. Otherwise, returns false .
boolean isRequestedSessionIdValid()	Returns true if the requested session ID is valid in the current session context.

Table: Various methods defined by HttpServletRequest

10.7.2. The HttpServletResponse Interface

The **HttpServletResponse** interface enables a servlet to formulate an HTTP response to a client. Several constants are defined. These correspond to the different status codes that can be assigned to an HTTP response. For example, SC_OK indicates that the HTTP request succeeded, and SC_NOT_FOUND indicates that the requested resource is not available. Several methods of this interface are summarized in table below.

Method	Description
void addCookie(Cookie cookie)	Adds <i>cookie</i> to the HTTP response.
boolean containsHeader(String field)	Returns true if the HTTP response header contains a field named <i>field</i> .
String encodeURL(String url)	Determines if the session ID must be encoded in the URL identified as <i>url</i> . If so, returns the modified version of <i>url</i> . Otherwise, returns <i>url</i> . All URLs generated by a servlet should be processed by this method.
String encodeRedirectURL(String url)	Determines if the session ID must be encoded in the URL identified as <i>url</i> . If so, returns the modified version of <i>url</i> . Otherwise, returns <i>url</i> . All URLs passed to sendRedirect() should be processed by this method.

Method	Description
<code>void sendError(int c)</code> throws <code>IOException</code>	Sends the error code <code>c</code> to the client.
<code>void sendError(int c, String s)</code> throws <code>IOException</code>	Sends the error code <code>c</code> and message <code>s</code> to the client.
<code>void sendRedirect(String url)</code> throws <code>IOException</code>	Redirects the client to <code>url</code> .
<code>void setDateHeader(String field, long msec)</code>	Adds <code>field</code> to the header with date value equal to <code>msec</code> (milliseconds since midnight, January 1, 1970, GMT).
<code>void setHeader(String field, String value)</code>	Adds <code>field</code> to the header with value equal to <code>value</code> .
<code>void setIntHeader(String field, int value)</code>	Adds <code>field</code> to the header with value equal to <code>value</code> .
<code>void setStatus(int code)</code>	Sets the status code for this response to <code>code</code> .

Table: Various methods defined by *HttpServletResponse*

10.7.3. The HttpSession Interface

The *HttpSession* interface enables a servlet to read and write the state information that is associated with an HTTP session. Several of its methods are summarized in Table below. All of these methods throw an *IllegalStateException* if the session has already been invalidated.

Method	Description
<code>Object getAttribute(String attr)</code>	Returns the value associated with the name passed in <code>attr</code> . Returns null if <code>attr</code> is not found.
<code>Enumeration getAttributeNames()</code>	Returns an enumeration of the attribute names associated with the session.
<code>long getCreationTime()</code>	Returns the time (in milliseconds since midnight, January 1, 1970, GMT) when this session was created.
<code>String getId()</code>	Returns the session ID.
<code>long getLastAccessedTime()</code>	Returns the time (in milliseconds since midnight, January 1, 1970, GMT) when the client last made a request for this session.
<code>void invalidate()</code>	Invalidates this session and removes it from the context.
<code>boolean isNew()</code>	Returns true if the server created the session and it has not yet been accessed by the client.
<code>void removeAttribute(String attr)</code>	Removes the attribute specified by <code>attr</code> from the session.
<code>void setAttribute(String attr, Object val)</code>	Associates the value passed in <code>val</code> with the attribute name passed in <code>attr</code> .

Table: The methods defined by *HttpSession*

10.7.4. The HttpSessionBindingListener Interface

The *HttpSessionBindingListener* interface is implemented by objects that need to be notified when they are bound to or unbound from an HTTP session. The methods that are invoked when an object is bound or unbound are

`void valueBound(HttpSessionBindingEvent e)`

`void valueUnbound(HttpSessionBindingEvent e)`

Here, `e` is the event object that describes the binding.

10.7.5. The Cookie Class

The *Cookie* class encapsulates a cookie. A **cookie** is stored on a client and contains state information. Cookies are valuable for tracking user activities. For example, assume that a user visits an online store. A cookie can save the user's name, address, and other information.

The user does not need to enter this data each time he or she visits the store. A servlet can write a cookie to a user's machine via the `addCookie( )` method of the *HttpServletResponse* interface. The data for that cookie is then included in the header of the HTTP response that is sent to the browser.

The names and values of cookies are stored on the user's machine. Some of the information that is saved for each cookie includes the following:

- The name of the cookie
- The value of the cookie
- The expiration date of the cookie
- The domain and path of the cookie

The expiration date determines when this cookie is deleted from the user's machine. If an expiration date is not explicitly assigned to a cookie, it is deleted when the current browser session ends. Otherwise, the cookie is saved in a file on the user's machine.

The domain and path of the cookie determine when it is included in the header of an HTTP request. If the user enters a URL whose domain and path match these values, the cookie is then supplied to the Web server. Otherwise, it is not. There is one constructor for Cookie. It has the signature shown here:

Cookie(String name, String value).

Here, the name and value of the cookie are supplied as arguments to the constructor.

Method	Description
Object clone()	Returns a copy of this object.
String getComment()	Returns the comment.
String getDomain()	Returns the domain.
int getMaxAge()	Returns the maximum age (in seconds).
String getName()	Returns the name.
String getPath()	Returns the path.
boolean getSecure()	Returns true if the cookie is secure. Otherwise, returns false .
String getValue()	Returns the value.
int getVersion()	Returns the version.
void setComment(String c)	Sets the comment to <i>c</i> .
void setDomain(String d)	Sets the domain to <i>d</i> .
void setMaxAge(int secs)	Sets the maximum age of the cookie to <i>secs</i> . This is the number of seconds after which the cookie is deleted.
void setPath(String p)	Sets the path to <i>p</i> .
void setSecure(boolean secure)	Sets the security flag to <i>secure</i> .
void setValue(String v)	Sets the value to <i>v</i> .
void setVersion(int v)	Sets the version to <i>v</i> .

Table: Methods defined by Cookie

10.8. Handling HTTP Requests and Responses

The **HttpServlet** class provides specialized methods that handle the various types of HTTP requests. A servlet developer typically overrides one of these methods. These methods are **doDelete()**, **doGet()**, **doHead()**, **doOptions()**, **doPost()**, **doPut()**, and **doTrace()**. The GET and POST requests are commonly used when handling form input.

10.8.1. Handling HTTP GET Requests

Here we will develop a servlet that handles an HTTP GET request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in **ColorGet.htm**, and a servlet is defined in **ColorGetServlet.java**. The HTML source code for **ColorGet.htm** is shown in the following listing. It defines a form that contains a select element and a submit button. Notice that the action parameter of the form tag specifies a URL. The URL identifies a servlet to process the HTTP GET request.

```
<html>
```

```
<body>
    <center>
        <form name="Form1" action="http://localhost:8080/servlets-
examples/servlet/ColorGetServlet">
            <B>Color:</B>
            <select name="color" size="1">
                <option value="Red">Red</option>
                <option value="Green">Green</option>
                <option value="Blue">Blue</option>
            </select>
            <br><br>
            <input type="submit" value="Submit">
        </form>
    </body>
</html>
```

The source code for *ColorGetServlet.java* is shown in the following listing. The **doGet()** method is overridden to process any HTTP GET requests that are sent to this servlet. It uses the **getParameter()** method of *HttpServletRequest* to obtain the selection that was made by the user. A response is then formulated.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorGetServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String color = request.getParameter("color");
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>The selected color is: ");
        pw.println(color);
        pw.close();
    }
}
```

After completing these steps, the browser will display the response that is dynamically generated by the servlet. One other point: Parameters for an HTTP GET request are included as part of the URL that is sent to the web server. Assume that the user selects the red option and submits the form. The URL sent from the browser to the server is

http://localhost:8080/servlets-examples/servlet/ColorGetServlet?color=Red

The characters to the right of the question mark are known as the *query string*.

10.8.2. Handling HTTP POST Requests

Here we will develop a servlet that handles an HTTP POST request. The servlet is invoked when a form on a web page is submitted. The example contains two files. A web page is defined in *ColorPost.htm*, and a servlet is defined in *ColorPostServlet.java*.

The HTML source code for *ColorPost.htm* is shown in the following listing. It is identical to *ColorGet.htm* except that the method parameter for the form tag explicitly specifies that the POST method should be used, and the action parameter for the form tag specifies a different servlet.

```
<html>
    <body>
        <center>
            <form name="Form1" method="post"
action="http://localhost:8080/servlets-
examples/servlet/ColorPostServlet">
                <B>Color:</B>
                <select name="color" size="1">
```

```

        <option value="Red">Red</option>
        <option value="Green">Green</option>
        <option value="Blue">Blue</option>
    </select>
    <br><br>
    <input type="submit" value="Submit">
</form>
</body>
</html>

```

The source code for *ColorPostServlet.java* is shown in the following listing. The **doPost()** method is overridden to process any HTTP POST requests that are sent to this servlet. It uses the **getParameter()** method of *HttpServletRequest* to obtain the selection that was made by the user. A response is then formulated.

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorPostServlet extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String color = request.getParameter("color");
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>The selected color is: ");
        pw.println(color);
        pw.close();
    }
}

```

Parameters for an HTTP POST request are not included as part of the URL that is sent to the web server. In this example, the URL sent from the browser to the server is

http://localhost:8080/servlets-examples/servlet/ColorPostServlet.

The parameter names and values are sent in the body of the HTTP request.

10.9. Session Tracking

HTTP is a stateless protocol. Each request is independent of the previous one. However, in some applications, it is necessary to save state information so that information can be collected from several interactions between a browser and a server. Sessions provide such a mechanism.

A session can be created via the **getSession()** method of *HttpServletRequest*. An *HttpSession* object is returned. This object can store a set of bindings that associate names with objects. The **setAttribute()**, **getAttribute()**, **getAttributeNames()**, and **removeAttribute()** methods of *HttpSession* manage these bindings. It is important to note that session state is shared among all the servlets that are associated with a particular client.

The following servlet illustrates how to use session state. The **getSession()** method gets the current session. A new session is created if one does not already exist. The **getAttribute()** method is called to obtain the object that is bound to the name "date". That object is a *Date* object that encapsulates the date and time when this page was last accessed. (Of course, there is no such binding when the page is first accessed.) A *Date* object encapsulating the current date and time is then created. The **setAttribute()** method is called to bind the name "date" to this object.

```

import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

```

```
public class DateServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    // Get the HttpSession object.
    HttpSession hs = request.getSession(true);
    // Get writer.
    response.setContentType("text/html");
    PrintWriter pw = response.getWriter();
    pw.print("<B>");
    // Display date/time of last access.
    Date date = (Date)hs.getAttribute("date");
    if(date != null) {
    pw.print("Last access: " + date + "<br>");
    }
    // Display current date/time.
    date = new Date();
    hs.setAttribute("date", date);
    pw.println("Current date: " + date);
}
}
```

When we first request this servlet, the browser displays one line with the current date and time information. On subsequent invocations, two lines are displayed. The first line shows the date and time when the servlet was last accessed. The second line shows the current date and time.

10.10. Java Server Page (JSP)

- *Java Server Pages (JSP)* is a technology for developing web pages that support dynamic content which helps developers *insert java code in HTML* pages by making use of special JSP tags, most of which start with `<%` and end with `%>`.
- A **Java Server** Pages component is a type of *Java servlet* that is designed to fulfill the role of a user interface for a Java web application.
- Web developers write JSPs as text files that combine HTML or XHTML code, XML elements, and embedded JSP actions and commands.
- Using JSP, we can collect input from users through web page forms, present records from a database or another source, and create web pages dynamically.
- JSP tags can be used for a variety of purposes, such as retrieving information from a database or *registering user preferences, accessing JavaBeans components, passing control between pages and sharing information between requests, pages* etc.

10.10.1. Why Use JSP?

Java Server Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.

- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having a separate CGI files.
- JSP are always compiled before it's processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- Java Server Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

- Finally, JSP is an integral part of J2EE, a complete platform for enterprise class applications. This means that JSP can play a part in the simplest applications to the most complex and demanding.

10.10.2. Advantages of JSP:

Following is the list of other advantages of using JSP over other technologies:

- **vs. Active Server Pages (ASP):** The advantages of JSP are two fold. **First**, the dynamic part is written in Java, not Visual Basic or other MS specific language, so it is more powerful and easier to use. **Second**, it is portable to other operating systems and non-Microsoft Web servers.
- **vs. Pure Servlets:** It is more convenient to write (and to modify!) regular HTML than to have plenty of `println` statements that generate the HTML.
- **vs. Server-Side Includes (SSI):** SSI is really only intended for simple inclusions, not for "real" programs that use form data, make database connections, and the like.
- **vs. JavaScript:** JavaScript can generate HTML dynamically on the client but can hardly interact with the web server to perform complex tasks like database access and image processing etc.
- **vs. Static HTML:** Regular HTML, of course, cannot contain dynamic information.

10.10.3. JSP Life Cycle

A JSP life cycle can be defined as the entire process from its creation till the destruction which is similar to a servlet life cycle with an additional step which is required to compile a JSP into servlet.

The following are the paths followed by a JSP

- Compilation
- Initialization
- Execution
- Cleanup

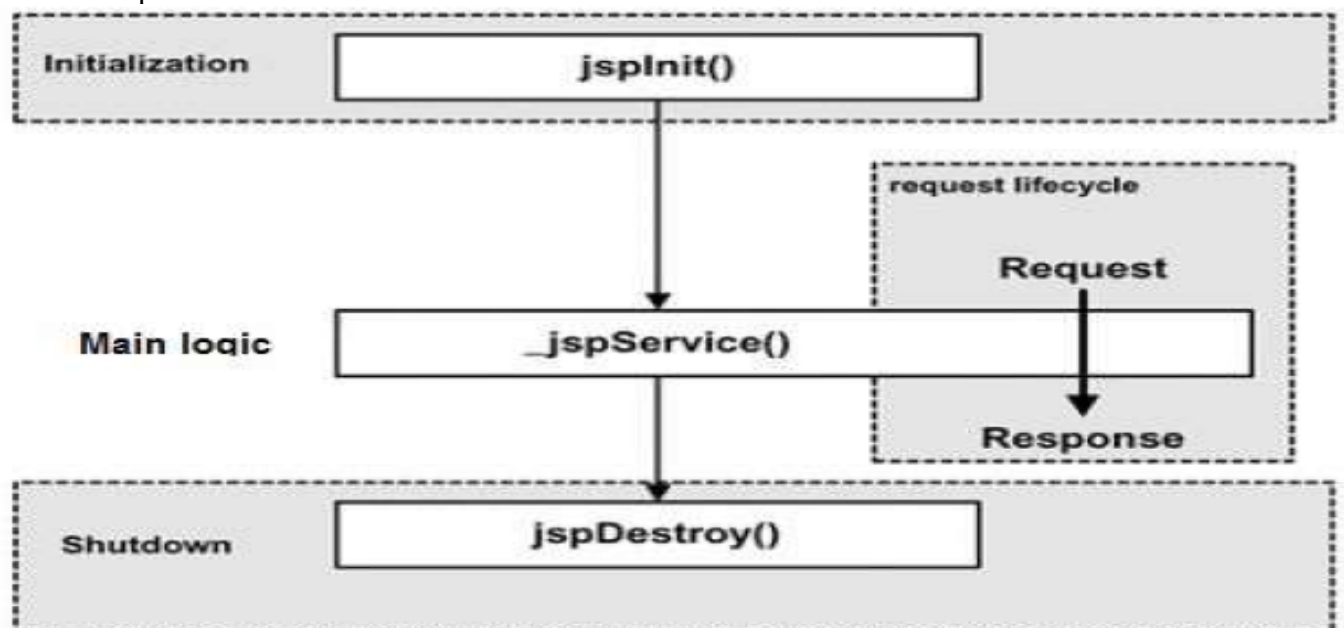


Fig: JSP Life Cycle

JSP Compilation

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page. The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP into a servlet.
- Compiling the servlet.

JSP Initialization

When a container loads a JSP it invokes the ***jspInit()*** method before servicing any requests. If we need to perform JSP-specific initialization, override the ***jspInit()*** method:

```
public void jspInit(){  
    // Initialization code...  
}
```

Typically initialization is performed only once and as with the servlet ***init*** method, we generally initialize database connections, open files, and create lookup tables in the ***jspInit*** method.

JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed. Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the ***_jspService()*** method in the JSP. The ***_jspService()*** method takes an **HttpServletRequest** and an **HttpServletResponse** as its parameters as follows:

```
void _jspService(HttpServletRequest request,  
HttpServletResponse response)  
{  
    // Service handling code...  
}
```

The ***_jspService()*** method of a JSP is invoked once per a request and is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods ie. GET, POST, DELETE etc.

JSP Cleanup

The destruction phase of the JSP life cycle represents when a JSP is being removed from use by a container. The ***jspDestroy()*** method is the JSP equivalent of the destroy method for servlets. We need to override ***jspDestroy*** to perform any cleanup, such as releasing database connections or closing open files.

The ***jspDestroy()*** method has the following form:

```
public void jspDestroy()  
{  
    // Your cleanup code goes here.  
}
```

10.10.4. JSP Syntax

A JSP Web page is created the same way you make an HTML file, except JSP code is added along with the HTML code. Additionally, instead of naming the file with an extension of **.HTM** or **.HTML**, the extension is **.JSP**.

The JSP elements are **declarations**, **expressions**, **scriptlets**, and **directives**. Their own tags identify all of these elements.

JSP element declaration

The declarations tag allows us to define **variables** and **methods**. The variables and methods are accessible from any **scriptlets** and **expressions** on the same page. Variable declarations are compiled as instance variables for a class that corresponds to the JSP page. Declarations are defined with the syntax

```
<%!
```

Declarations

```
%>
```

For example, the following defines an instance variable named **count** and a method named **incrementCount()** that increments the count variable:

```
<%!
```

```
private int count = 0;
private void incrementCount()
{
    count++;
}
%>
```

JSP Expression

We can access variables defined in declarations with expression. The syntax to embed an expression is as follows:

```
<%=
Expression
%>
```

Expressions are embedded directly into the HTML. The Web browser will display the value of the expression in place of the tag. For example, we can output the value of the **count** variable in bold type with the following piece of HTML:

The value of count is <%= count %>

JSP Scriptlet

Blocks of Java code can be embedded in a *scriptlet*. The syntax for a *scriptlet* is as follows:

```
<%
Java code
%>
```

If you wish to output HTML within a *scriptlet*, then this is done using **out.println()**, which is used in the same manner as **System.out.println()**. The variable **out** is already defined for us and is of type **javax.servlet.jsp.JspWriter**. Also note that **System.out.println()** will output to the console, which is useful for debugging purposes, while **out.println()** will output to the browser. The following *scriptlet* invokes the **incrementCount()** method and then outputs the value in count :

```
<%
out.println("The counter's value is " + count + "<br />");
incrementCount();
%>
```

Following is a JSP page with a *declaration*, *expression*, and *scriptlet* that outputs text inside a header tag from levels 1 to 6. The identifier LASTLEVEL is declared as the last heading level that is to be displayed. LASTLEVEL is modified by static final, because it is intended as a constant. A loop inside the *scriptlet* outputs sample text for each level.

```
<html>
<title>
    Displaying Heading Tags with JSP
</title>
<body>
    <%!
        private static final int LASTLEVEL = 6;
    %>
    <p>
        This page uses JSP to display Heading Tags from
        Level 1 to Level <%= LASTLEVEL %>
    </p>
    <%
        int i;
        for (i = 1; i <= LASTLEVEL; i++)
        {
```

```

        out.println("<H" + i + ">" +
        "This text is in Heading Level " + i +
        "</H" + i + ">");
    }
    %>
</body>
</html>

```

JSP Directives

Finally, let us introduce one more JSP tag, the **directive**. In general terms, directives instruct the compiler how to process a JSP program. Examples include the definition of our own tags, including the source code of other files, and importing packages. The syntax for directives is as follows:

```

<%@
page import="java.util.*,java.sql.*"
%>

```

10.10.5. Handling Parameters in JSP

To make a JSP page more interactive, we can read and process the data entered in an HTML form. One way to read these values is to call the **request.getParameter()** method. This method takes a String parameter as input that identifies the name of an HTML form element and returns the value entered by the user for that element on the form. For example, if there is a *textbox* named *AuthorID*, then we can retrieve the value entered in that textbox with the following **scriptlet** code:

String value = request.getParameter("AuthorID");

If the user leaves the field blank, then *getParameter* returns an empty string. The following Html file uses a *EditURL.jsp* in order to process request.

```

<html>
    <head>
        <title>Change Author's URL</title>
    </head>
    <body>
        <h1>Change Author's URL</h1>
        <p>
            Enter the ID of the author you would like to change
            along with the new URL.
        </p>
        <form ACTION = "EditURL.jsp" METHOD = POST>
            Author ID:
            <input TYPE = "TEXT" NAME = "AuthorID" VALUE = "" SIZE = "4"
            MAXLENGTH = "4">
            <br/>
            New URL:
            <input TYPE = "TEXT" NAME = "URL"
            VALUE = "http://" SIZE = "40" MAXLENGTH = "200">
            <p>
                INPUT TYPE="SUBMIT" VALUE="Submit">
            </p>
        </form>
    </body>
</html>

```

The following JSP program echoes back the data entered by the user in above html. The name of the JSP file must match the value supplied for the ACTION tag of the form.

```

<html>

```

```
<title>Edit URL: Echo submitted values</title>
<body>
    <h2>Edit URL</h2>
    <p>
        This version of EditURL.jsp simply echoes back to the
        user the values that were entered in the textboxes.
    </p>
    <%
        String url = request.getParameter("URL");
        String stringID = request.getParameter("AuthorID");
        int author_id = Integer.parseInt(stringID);
        out.println("The submitted author ID is: " + author_id);
        out.println("<br/>");
        out.println("The submitted URL is: " + url);
    %>
</body>
</html>
```

10.10.6. JSP Implicit Objects

JSP Implicit Objects are the Java objects that the JSP Container makes available to developers in each page and developer can call them directly without being explicitly declared. JSP Implicit Objects are also called pre-defined variables. JSP supports nine Implicit Objects which are listed below:

Object	Description
request	This is the HttpServletRequest object associated with the request.
response	This is the HttpServletResponse object associated with the response to the client.
out	This is the PrintWriter object used to send output to the client.
session	This is the HttpSession object associated with the request.
application	This is the ServletContext object associated with application context.
config	This is the ServletConfig object associated with the page.
pageContext	This encapsulates use of server-specific features like higher performance JspWriters .
page	This is simply a synonym for this, and is used to call the methods defined by the translated servlet class.
Exception	The Exception object allows the exception data to be accessed by designated JSP.

1. The **request object** is an instance of a **javax.servlet.http.HttpServletRequest** object. Each time a client requests a page the JSP engine creates a new object to represent that request. The **request** object provides methods to get HTTP header information including form data, cookies, HTTP methods etc.
2. The **response object** is an instance of a **javax.servlet.http.HttpServletResponse** object. Just as the server creates the request object, it also creates an object to represent the response to the client. The response object also defines the interfaces that deal with creating new HTTP headers. Through this object the JSP programmer can add new cookies or date stamps, HTTP status codes etc.
3. The **out** implicit object is an instance of a **javax.servlet.jsp.JspWriter** object and is used to send content in a response. The initial **JspWriter** object is instantiated differently depending on whether the page is buffered or not. Buffering can be easily turned off by using the buffered='false' attribute of the page directive. The **JspWriter** object contains most of the same methods as the **java.io.PrintWriter** class. However, **JspWriter** has some additional methods designed to deal with

buffering. Unlike the *PrintWriter* object, *JspWriter* throws *IOExceptions*.

4. The **session object** is an instance of ***javax.servlet.http.HttpSession*** and behaves exactly the same way that session objects behave under Java Servlets. The session object is used to track client session between client requests.
5. The **application object** is direct wrapper around the ***ServletContext*** object for the generated Servlet and in reality an instance of a ***javax.servlet.ServletContext*** object. This object is a representation of the JSP page through its entire lifecycle. This object is created when the JSP page is initialized and will be removed when the JSP page is removed by the ***jspDestroy()*** method.
6. The **config object** is an instantiation of ***javax.servlet.ServletConfig*** and is a direct wrapper around the ***ServletConfig*** object for the generated servlet. This object allows the JSP programmer access to the Servlet or JSP engine initialization parameters such as the paths or file locations etc.
7. The **pageContext object** is an instance of a ***javax.servlet.jsp.PageContext*** object. The **pageContext** object is used to represent the entire JSP page. This object is intended as a means to access information about the page while avoiding most of the implementation details. This object stores references to the request and response objects for each request. The *application*, *config*, *session*, and *out* objects are derived by accessing attributes of this object.
8. **Page** object is an actual reference to the instance of the page. It can be thought of as an object that represents the entire JSP page. The page object is really a direct synonym for the **this** object.
9. The **exception** object is a wrapper containing the exception thrown from the previous page. It is typically used to generate an appropriate response to the error condition.

10.10.7. JavaServer Pages Standard Tag Library (JSTL)

JavaServer Pages Standard Tag Library (JSTL) is a collection of useful JSP tags which encapsulates core functionality common to many JSP applications. JSTL has support for *common*, *structural tasks* such as *iteration and conditionals*, *tags for manipulating XML documents*, *internationalization tags*, and *SQL tags*. It also provides a framework for integrating existing custom tags with JSTL tags.

The JSTL tags can be classified, according to their functions, into following JSTL tag library groups that can be used when creating a JSP page:

- Core Tags
- Formatting tags
- SQL tags
- XML tags
- JSTL Functions

Core Tags

The core group of tags is the most frequently used JSTL tags. Following is the syntax to include JSTL Core library in our JSP:

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

Tag	Description
<u><c:out></u>	Like <%= ... >, but for expressions.
<u><c:set></u>	Sets the result of an expression evaluation in a 'scope'
<u><c:remove></u>	Removes a scoped variable (from a particular scope, if specified).
<u><c:catch></u>	Catches any Throwable that occurs in its body and optionally exposes it.
<u><c:if></u>	Simple conditional tag which evaluates its body if the supplied condition is true.
<u><c:choose></u>	Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise>
<u><c:when></u>	Subtag of <choose> that includes its body if its condition evaluates to 'true'.
<u><c:otherwise></u>	Subtag of <choose> that follows <when> tags and runs only if all of the prior conditions evaluated to 'false'.
<u><c:import></u>	Retrieves an absolute or relative URL and exposes its contents to either the page, a String in 'var', or a Reader in 'varReader'.
<u><c:forEach></u>	The basic iteration tag, accepting many different collection types and supporting subsetting and other functionality .
<u><c:forEachTokens></u>	Iterates over tokens, separated by the supplied delimiters.
<u><c:param></u>	Adds a parameter to a containing 'import' tag's URL.
<u><c:redirect></u>	Redirects to a new URL.
<u><c:url></u>	Creates a URL with optional query parameters

Formatting tags

The JSTL formatting tags are used to format and display text, the date, the time, and numbers for internationalized Web sites. Following is the syntax to include formatting library in our JSP:

`<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>`

Tag	Description
<u><fmt:formatNumber></u>	To render numerical value with specific precision or format.
<u><fmt:parseNumber></u>	Parses the string representation of a number, currency, or percentage.
<u><fmt:formatDate></u>	Formats a date and/or time using the supplied styles and pattern
<u><fmt:parseDate></u>	Parses the string representation of a date and/or time
<u><fmt:bundle></u>	Loads a resource bundle to be used by its tag body.
<u><fmt:setLocale></u>	Stores the given locale in the locale configuration variable.
<u><fmt:setBundle></u>	Loads a resource bundle and stores it in the named scoped variable or the bundle configuration variable.
<u><fmt:timeZone></u>	Specifies the time zone for any time formatting or parsing actions nested in its body.
<u><fmt:setTimeZone></u>	Stores the given time zone in the time zone configuration variable
<u><fmt:message></u>	To display an internationalized message.
<u><fmt:requestEncoding></u>	Sets the request character encoding

SQL tags

The JSTL SQL tag library provides tags for interacting with relational databases (RDBMSs) such as Oracle, MySQL, or Microsoft SQL Server. Following is the syntax to include JSTL SQL library in our JSP:

```
<%@ taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql" %>
```

Tag	Description
<u><sql:setDataSource></u>	Creates a simple DataSource suitable only for prototyping
<u><sql:query></u>	Executes the SQL query defined in its body or through the sql attribute.
<u><sql:update></u>	Executes the SQL update defined in its body or through the sql attribute.
<u><sql:param></u>	Sets a parameter in an SQL statement to the specified value.
<u><sql:dateParam></u>	Sets a parameter in an SQL statement to the specified java.util.Date value.
<u><sql:transaction ></u>	Provides nested database action elements with a shared Connection, set up to execute all statements as one transaction.

XML tags

The JSTL XML tags provide a JSP-centric way of creating and manipulating XML documents. Following is the syntax to include JSTL XML library in our JSP. The JSTL XML tag library has custom tags for interacting with XML data. This includes parsing XML, transforming XML data, and flow control based on XPath expressions.

```
<%@ taglib prefix="x" uri="http://java.sun.com/jsp/jstl/xml" %>
```

Tag	Description
<u><x:out></u>	Like <%= ... >, but for XPath expressions.
<u><x:parse></u>	Use to parse XML data specified either via an attribute or in the tag body.
<u><x:set ></u>	Sets a variable to the value of an XPath expression.
<u><x:if ></u>	Evaluates a test XPath expression and if it is true, it processes its body. If the test condition is false, the body is ignored.
<u><x:forEach></u>	To loop over nodes in an XML document.
<u><x:choose></u>	Simple conditional tag that establishes a context for mutually exclusive conditional operations, marked by <when> and <otherwise>
<u><x:when ></u>	Subtag of <choose> that includes its body if its expression evaluates to 'true'
<u><x:otherwise ></u>	Subtag of <choose> that follows <when> tags and runs only if all of the prior conditions evaluated to 'false'
<u><x:transform ></u>	Applies an XSL transformation on a XML document
<u><x:param ></u>	Use along with the transform tag to set a parameter in the XSLT stylesheet

JSTL Functions

JSTL includes a number of standard functions, most of which are common string manipulation functions. Following is the syntax to include JSTL Functions library in our JSP:

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>
```

Function	Description
<u>fn:contains()</u>	Tests if an input string contains the specified substring.
<u>fn:containsIgnoreCase()</u>	Tests if an input string contains the specified substring in a case insensitive way.
<u>fn:endsWith()</u>	Tests if an input string ends with the specified suffix.
<u>fn:escapeXml()</u>	Escapes characters that could be interpreted as XML markup.
<u>fn:indexOf()</u>	Returns the index withing a string of the first occurrence of a specified substring.
<u>fn:join()</u>	Joins all elements of an array into a string.
<u>fn:length()</u>	Returns the number of items in a collection, or the number of characters in a string.
<u>fn:replace()</u>	Returns a string resulting from replacing in an input string all occurrences with a given string.
<u>fn:split()</u>	Splits a string into an array of substrings.
<u>fn:startsWith()</u>	Tests if an input string starts with the specified prefix.
<u>fn:substring()</u>	Returns a subset of a string.
<u>fn:substringAfter()</u>	Returns a subset of a string following a specific substring.
<u>fn:substringBefore()</u>	Returns a subset of a string before a specific substring.
<u>fn:toLowerCase()</u>	Converts all of the characters of a string to lower case.
<u>fn:toUpperCase()</u>	Converts all of the characters of a string to upper case.
<u>fn:trim()</u>	Removes white spaces from both ends of a string.